Generate and Filter: A GNN-Based Approach for Graph Anomaly Detection

Mengyu Li, Yonghao Liu, *Member, IEEE*, Ximing Li, *Member, IEEE*, Lan Huang, Fausto Giunchiglia, Yanchun Liang, Xiaoyue Feng, *Member, IEEE*, and Renchu Guan, *Senior Member, IEEE*

***Abstract*—Graph anomaly detection (GAD) aims to distinguish anomalies from the majority of normal nodes in graph-structured data. Due to its extensive real-world applications, GAD has garnered increasing attention from both academia and industry. Recently, Graph Neural Networks (GNNs) have been integrated into GAD frameworks, yielding promising results by effectively characterizing structural information. However, existing GNN-based methods suffer from several critical limitations: (I) the difficulty of learning discriminative representations for anomalies in the feature space; (II) the structural sparsity caused by the lack of essential connections between anomalies; and (III) the camouflage effect resulting from redundant edges between anomalies and normal nodes. To address these challenges, we propose a novel framework, Generate and Filter graph learning for Graph Anomaly Detection (GFGAD). Specifically, GFGAD first generates a diverse set of synthetic anomalies with enriched feature and structural information to balance the data distribution. Subsequently, these generated anomalies are strategically connected to original ones to compensate for missing structural patterns, while a filtering mechanism is employed to eliminate redundant connections and mitigate camouflage. Extensive experiments on several benchmark datasets demonstrate that GFGAD significantly outperforms state-of-the-art baselines.**

***Index Terms*—Graph neural networks, anomaly detection, edge filtering, imbalanced learning.**

I. Introduction

GRAPH Anomaly Detection (GAD) aims to identify the sparse set of anomalous nodes within graph-structured data that deviate significantly from the majority. As a critical task, GAD has garnered substantial interest from both industry and academia, primarily because graph-structured data are ubiquitous and excel at capturing complex relationships between diverse entities. Its significance is further underscored by a wide array of applications, ranging from cybersecurity [1] and financial fraud detection [2] to online review management [3]. Recently, driven by the rapid maturation of Graph Neural Networks (GNNs) [4], [5], numerous GAD frameworks have integrated GNN techniques to achieve competitive performance [3]. The core intuition behind these approaches is to leverage GNNs to learn expressive node embeddings that facilitate the separation of anomalies in a high-dimensional latent space. Compared to traditional graph-based methods, GNN-based paradigms offer the advantage of end-to-end training without the need for manual feature engineering. Currently, the prevailing GNN-based approaches formulate GAD as a semi-supervised binary classification task, where the model is trained on a limited set of labeled nodes and evaluated on the remaining nodes [6].

Despite their empirical success, existing GNN-based GAD models suffer from several inherent limitations. *First*, the extreme scarcity of abnormal instances often leads to a severe **representation bias**. Models such as FdGars [8] and GAL [9] struggle to learn discriminative features because the class imbalance skews the learning process toward the majority, thereby compressing the anomaly representation space [10] and hindering the capture of outlier-specific characteristics [11]. As illustrated in Fig. 1, this often results in poorly separated embeddings in real-world networks. *Second*, the **structural sparsity** inherent in anomalous nodes poses a significant challenge. Unlike normal nodes, anomalies in scenarios such as financial fraud often operate in isolation to evade collective detection [3], a phenomenon also observed in the Enron and Flickr datasets (Fig. 1). Since GNNs typically function as low-pass filters that smooth features across adjacent nodes [12], this lack of "anomaly-to-anomaly" connectivity (i.e., low homophily) prevents models from effectively mining underlying patterns through topological proximity. *Third*, the issue is further exacerbated by **feature contamination** arising from adversarial camouflage [13]. For instance, social network fraudsters often deliberately connect to benign entities to appear legitimate [14], creating redundant and noisy edges. These cross-class connections lead to the "smearing" of anomaly features during GNN neighborhood aggregation, rendering the learned embeddings indistinguishable from normal ones [15].

Received 14 May 2025; revised 14 July 2026; accepted 6 August 2026. Date of publication 17 August 2026; date of current version 5 October 2026. The work of Fausto Giunchiglia's was supported by European Union's Horizon 2020 FET Proactive Project under Grant 823783. This work was supported in part by the National Natural Science Foundation of China under Grant 62372209, Grant 62372494, and Grant 62172187, and in part by Guangdong Quality Engineering Project under Grant 2024001. Recommended for acceptance by B. Rosenhahn. *(Mengyu Li and Yonghao Liu equally contributed to this work.) (Corresponding authors: Xiaoyue Feng; Renchu Guan.)*

Mengyu Li, Yonghao Liu, Ximing Li, Lan Huang, Xiaoyue Feng, and Renchu Guan are with the Key Laboratory of Symbolic Computation and Knowledge Engineering of the Ministry of Education, College of Computer Science and Technology, Jilin University, Changchun 130012, China (e-mail: mengyul21@mails.jlu.edu.cn; yonghao20@mails.jlu.edu.cn; liximing86@gmail.com; huanglan@jlu.edu.cn; fengxy@jlu.edu.cn; guanrenchu@jlu.edu.cn).

Fausto Giunchiglia is with the University of Trento, 38122 Trento, Italy (e-mail: fausto.giunchiglia@unitn.it).

Yanchun Liang is with the Zhuhai Laboratory of the Key Laboratory of Symbolic Computation and Knowledge Engineering of the Ministry of Education, Zhuhai College of Science and Technology, Zhuhai 519041, China (e-mail: ycliang@jlu.edu.cn).

Digital Object Identifier 10.1109/TPAMI.2026.3724665

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

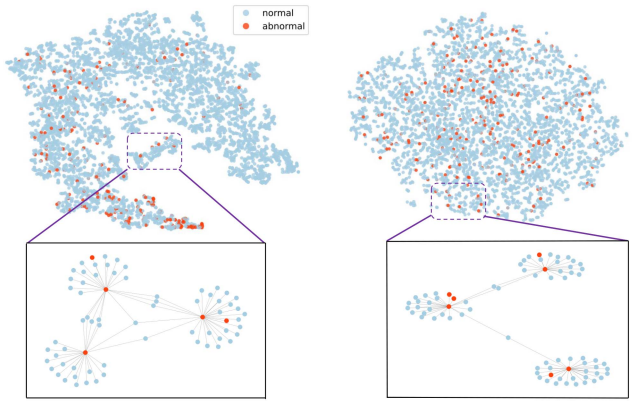


Fig. 1. Visualization of node embeddings for abnormal and normal nodes learned by GAL on the Enron dataset (left) and the Flickr dataset (right). The original data is fed into GAL [7] without any preprocessing, and the resulting node embeddings are visualized. The lower panels present magnified views, with some non-essential connections between normal and abnormal nodes omitted for improved clarity.

Consequently, as shown in the zoomed-in views of Fig. 1, maintaining the discriminative power of GNNs amidst such structural noise remains a non-trivial task.

In this work, we focus on identifying anomalous nodes in real-world graph-structured data and formulate GAD as a binary classification task. To address the aforementioned challenges, we propose GFGAD, a novel "Generate and Filter" graph learning framework. The core philosophy of GFGAD is to leverage a generative adversarial mechanism to synthesize a substantial volume of diverse anomalies, thereby balancing the data distribution and preventing the model from being dominated by the majority class during training. Specifically, a neural network-based generator yields a supplementary attributed graph containing synthetic anomalies. To bridge the structural gap, we establish essential links between these generated anomalies and existing abnormal nodes in the original graph, as these topological connections help reveal underlying label semantics. Following the adversarial paradigm, a GNN-based discriminator is introduced to distinguish synthetic samples from real data.

To ensure the fidelity of these samples, we further incorporate an auxiliary GNN-based detector that cooperates with the generator to drive synthetic nodes toward the anomalous distribution while remaining distinct from normal patterns. Furthermore, to mitigate feature contamination and suppress camouflage, we devise a graph neighbor filter to refine the balanced graph. By selectively filtering edges that connect nodes with disparate feature representations, this mechanism rectifies confusing links and enhances the homophily of the graph. Consequently, GFGAD constructs a high-quality, balanced attributed graph where "birds of a feather flock together," significantly facilitating the learning of discriminative GNN embeddings.

In summary, our principal contributions are as follows:

- We propose GFGAD, a novel generative adversarial framework for GAD that integrates a generator, a GNN-based discriminator, an auxiliary detector, and a graph neighbor filter to collectively enhance detection performance.
- We introduce a joint generative-and-filtering strategy that synthesizes informative abnormal nodes and establishes essential topological links while concurrently pruning redundant camouflaged edges to reinforce graph homophily.
- We conduct extensive evaluations on seven real-world benchmark datasets. The experimental results demonstrate that GFGAD consistently and significantly outperforms state-of-the-art baselines across various metrics.

II. Related Work

A. Graph Neural Networks

Due to their exceptional representational power, GNNs have garnered significant attention as a premier deep learning framework for graph-structured data [16]. GNNs have been successfully deployed across a wide spectrum of applications, including social network analysis [17], [18], text classification [19], [20], question answering systems [21], [22], and fraud detection [3], [12]. Most contemporary GNNs adhere to the message-passing paradigm, where the representation of a target node is iteratively updated by aggregating information from its local neighborhood. As a seminal work, Graph Convolutional Networks (GCNs) [4] defined graph convolutions in the spectral domain using a first-order approximation. Subsequent studies have further explored the theoretical foundations of GCNs, analyzing key factors that influence their stability and generalization [23]. Furthermore, various enhanced GNN architectures [24], [25] have been proposed to extract discriminative node embeddings for complex downstream tasks. For instance, MCGRL [25] captures multi-granularity information across diverse topological structures to enrich node representations. However, while these methods excel at general network representation learning, they are often sub-optimal for GAD. As previously discussed, the inherent message-passing mechanism typically acts as a low-pass filter, which may inadvertently dilute the anomalous features by over-smoothing them with those of normal neighbors.

B. GNN-Based Anomaly Detection

Early GAD attempts primarily relied on manual feature engineering, which often suffered from labor-intensive processes and limited generalization [26]. With the advent of deep graph learning, contemporary methods have shifted toward designing sophisticated GNN encoders to learn label-guided node embeddings for outlier discovery. These models have been extensively applied to critical domains such as financial fraud detection [2], [27] and review fraud identification [3], [8]. For instance, CARE-GNN [3] introduces a reinforced neighbor selector to counteract the camouflage of fraudsters, while PC-GNN [27] develops a label-balanced sampler to construct subgraphs for more robust representation learning. To capture diverse spectral patterns, AMNet [28] adaptively filters low-frequency and high-frequency graph signals. Similarly, SparseGAD [29] mitigates noise by sparsifying the graph structure, thereby unveiling the complex homophily and heterophily relationships between node pairs. More recently, PMP [30] proposes a partitioning

message-passing paradigm with category-specific aggregation functions to prevent the majority class from dominating the gradients. CONSISGAD [31] further enhances discriminative power by leveraging learnable data augmentation for consistency training and exploiting the distinct homophily distributions of normal versus anomalous nodes. Additionally, some self-supervised approaches [32], [33] have been proposed for unsupervised GAD; however, these operate under a different learning paradigm than our semi-supervised setting. Crucially, while the aforementioned models tackle GAD by refining the internal GNN architecture, they typically overlook the potential of directly modulating the underlying data distribution. Unlike these methods, our work focuses on rectifying the inherent class imbalance and structural gaps through a generative-and-filtering strategy.

C. Generative Adversarial Networks for Graphs

Generative Adversarial Networks (GANs) [34] have revolutionized machine learning, with their impact extending across diverse research domains. A standard GAN comprises a generator and a discriminator that engage in a minimax game to mutually enhance their representative capabilities through adversarial learning. Recently, this philosophy has been successfully adapted to graph-structured data to refine node representations [35], [36], [37]. For instance, GraphGAN [35] employs a generator to approximate the underlying connectivity distribution of vertices, while the discriminator distinguishes between true neighbors and sampled ones. NetGAN [37] mimics random walks to capture and replicate complex graph topologies. In the domain of molecular informatics, MolGAN [38] integrates a reward network with the GAN framework to synthesize molecular graphs with desired chemical properties. Inspired by these advancements, several GAN-based solutions have emerged for GAD. Specifically, GAAN [39] utilizes a BiGAN architecture with an additional encoder to map both real and synthetic nodes into a latent space, capturing structural information via covariance matrices. STGAD [40] incorporates graph convolutional gated recurrent units to facilitate the separate learning of spatial-temporal node features within the adversarial process. However, most existing models strictly adhere to the conventional two-player GAN paradigm. In contrast, to better characterize the subtle deviations of anomalies and synthesize more informative fake instances, we propose a triadic architecture that integrates a generator, a discriminator, and a specialized detector, specifically designed to enhance the discriminative quality of the generated anomalous distribution.

III. Definition and Preliminary

In this section, we define the GAD problem and present the basics of GNN in detail. To better understand our model, the important symbols used in this work are listed in Table I, and other special symbols are explained in their corresponding sections.

TABLE I
Descriptions of Important Symbols

Symbols	Description
$\mathcal{G}, \mathcal{V}, \mathcal{E}$	original network, node set, and edge set
$\mathbf{X}, \mathbf{A}, \mathcal{Y}$	original feature matrix, adjacency matrix, and label set
$\mathcal{V}_{\text{ab}}, \mathcal{V}_{\text{nor}}, \mathcal{V}_{\text{syn}}$	abnormal, normal, and synthetic node sets
$n_{\text{ab}}, n_{\text{nor}}, n_{\text{syn}}$	numbers of abnormal, normal, and synthetic nodes
$\tilde{\mathcal{G}}, \tilde{\mathcal{V}}, \tilde{\mathcal{E}}$	balanced network, merged node set, and edge set
$\tilde{\mathbf{X}}, \tilde{\mathbf{A}}, \tilde{\mathcal{Y}}$	merged feature matrix, adjacency matrix, and label set
$\mathbf{G}, \mathbf{D}, \mathbf{T}$	generator, discriminator, and detector
$\tilde{\mathbf{S}}$	synthetic connectivity distribution
η	threshold for the synthetic connectivity distribution $\tilde{\mathbf{S}}$
α, λ	generation ratio, distance threshold
$\mathbf{W}$	learnable weight matrices
Ω	Gaussian random noise space

A. Problem Definition

Suppose we are given an attributed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$, where $\mathcal{V}$ is the set of nodes $\{v_1, v_2, \ldots, v_n\}$ and $\mathcal{E}$ is the set of edges $\{e_1, e_2, \ldots, e_m\}$. $\mathbf{X} \in \mathbb{R}^{n \times d}$ denotes the node feature matrix, where an attribute vector $\mathbf{x}_i \in \mathbb{R}^d$ is assigned to each node. $\mathbf{A} = \{0, 1\}^{n \times n}$ is an adjacency matrix denoting the graph structure. Let $\mathcal{Y} = \{y_1, y_2, \ldots, y_n\}$ be the set of node labels, with $y_i \in \{0, 1\}$ indicating whether node v_i is normal ($y_i = 0$) or anomalous ($y_i = 1$).

Graph Anomaly Detection: In the semi-supervised GAD setting, labels are only partially available for a subset of nodes $\mathcal{V}_{\text{train}} \subset \mathcal{V}$, denoted as $\mathcal{Y}_{\text{train}}$. The objective is to learn a predictive mapping $\mathcal{H} : \mathcal{V} \to \{0, 1\}$ by leveraging both the available labels $\mathcal{Y}_{\text{train}}$ and the graph's structural and attribute information. Formally, the problem is to train a classifier $\mathcal{H}$ that effectively identifies anomalies within the unlabeled set $\mathcal{V}_{\text{test}} = \mathcal{V} \setminus \mathcal{V}_{\text{train}}$, which can be formulated as:

$$\mathcal{H} : (\mathcal{G}, \mathcal{Y}_{\text{train}}) \to \hat{\mathcal{Y}}_{\text{test}}, \tag{1}$$

where $\hat{\mathcal{Y}}_{\text{test}}$ denotes the predicted labels for the unlabeled nodes.

We define the training set as $\mathcal{V}_{\text{train}} = \mathcal{V}_{\text{nor}} \cup \mathcal{V}_{\text{ab}}$, where $\mathcal{V}_{\text{nor}}$ denotes the normal nodes and $\mathcal{V}_{\text{ab}}$ represents the abnormal nodes. Additionally, we denote the training nodes in the generated balanced graph as $\mathcal{V}_{\text{train}} \cup \mathcal{V}_{\text{syn}}$, where $\mathcal{V}_{\text{syn}}$ represents the set of synthesized nodes.

B. The Basics of GNNs

Most contemporary GNNs adhere to a message-passing framework characterized by a neighborhood aggregation mechanism. Under this paradigm, the representation of a given node is iteratively refined by integrating features from its local neighborhood, a process essential for encoding complex structural dependencies. The framework proposed in this work is architecturally flexible and compatible with various GNN backbones. By default, we implement our components using GCNs [4]. GCNs employ a first-order approximation of spectral graph convolutions to aggregate neighborhood information. Formally, the layer-wise propagation rule is defined as:

$$\mathbf{H}^{(k)} = \sigma\left(\widehat{\mathbf{A}}\mathbf{H}^{(k-1)}\mathbf{W}^{(k-1)}\right), \tag{2}$$

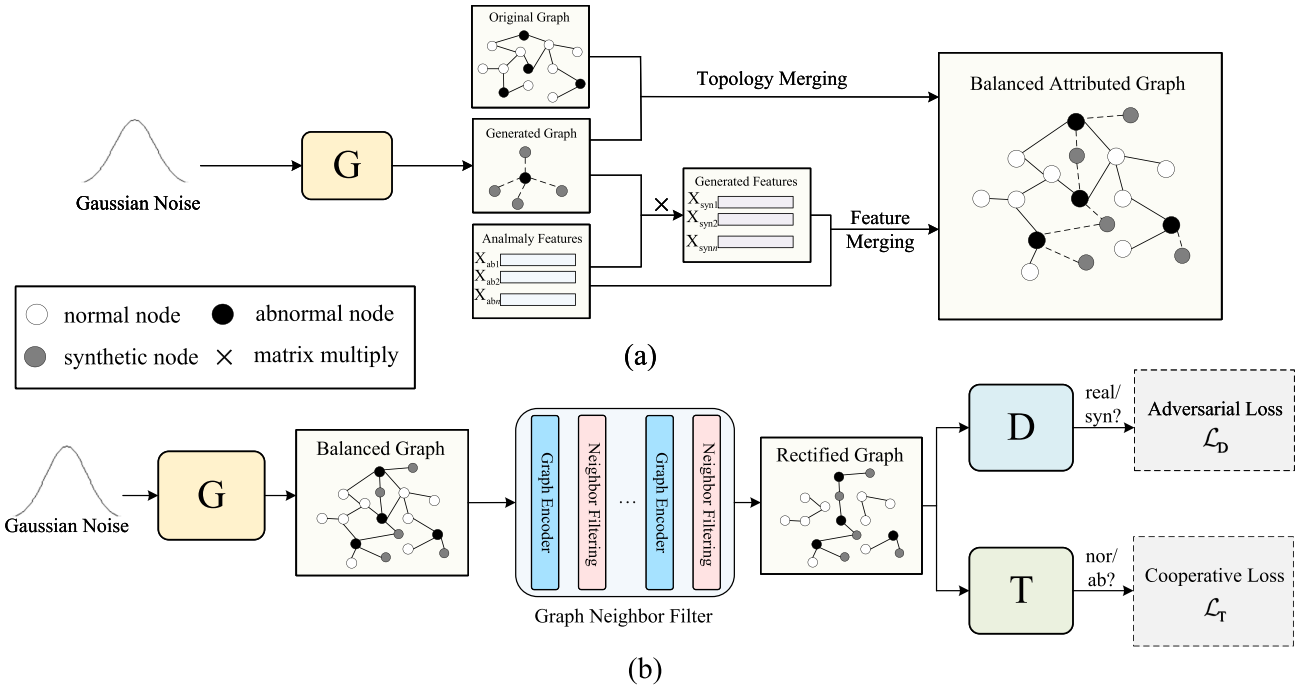


Fig. 2. The overall architecture of GFGAD. (a) Anomaly Generation: G synthesizes informative anomalies (gray) by aggregating features from existing outliers (black) and establishing new topological links to form a balanced attributed graph. (b) Graph Refinement and Adversarial Learning: The graph neighbor filter prunes redundant connections based on L_2 distance to mitigate camouflage. The resulting rectified graph is used to train the discriminator D and auxiliary detector T via a triadic adversarial strategy, where T and G cooperate to enhance the discriminative quality of synthetic nodes.

where $\widehat{\mathbf{A}} = (\mathbf{D}+\mathbf{I})^{-1/2}(\mathbf{A}+\mathbf{I})(\mathbf{D}+\mathbf{I})^{-1/2}$ is the normalized symmetric adjacency matrix with self-loops, in which $\mathbf{D}_{ii} = \sum_j \mathbf{A}_{ij}$ is the corresponding diagonal degree matrix, and $\mathbf{I}$ is the identity matrix. $\mathbf{H}^{(0)} = \mathbf{X}$ is the initial node feature matrix, and $\mathbf{W}^{(k)}$ is the layer-specific learnable weight.

IV. Method

In this section, we present the technical details of the proposed GFGAD framework. The overall architecture, as illustrated in Fig. 2, is designed to address the challenges of class imbalance, structural sparsity, and adversarial camouflage through a "Generate and Filter" paradigm. GFGAD consists of four synergistically integrated components: a Generator (G), a Graph Neighbor Filter, a Discriminator (D), and an Auxiliary Detector (T).

As depicted in Fig. 2(a), the process begins with the anomaly generation stage. The generator G synthesizes a diverse set of informative anomalies by aggregating and transforming the features of existing outliers. Unlike conventional generative models, G also establishes strategic topological links between synthetic and real anomalies, thereby constructing a balanced attributed graph that alleviates the inherent data scarcity and structural isolation of abnormal nodes.

Following the construction of the balanced graph, the framework proceeds to the structural refinement and adversarial learning stage, shown in Fig. 2(b). To counteract the camouflage effect where anomalies mimic benign patterns, the graph neighbor filter adaptively prunes redundant or noisy connections by evaluating the L_2 distance between node embeddings. The resulting rectified graph serves as a high-quality manifold for training the remaining modules. Specifically, the discriminator D engages in a minimax game with G to ensure the fidelity of synthetic data, while the auxiliary detector T cooperates with G to refine the discriminative boundaries. Through this triadic optimization, GFGAD effectively learns robust representations for GAD tasks. In the following subsections, we delve into the mathematical formulations and operational mechanisms of each component.

A. Generator

To address the inherent class imbalance and structural sparsity of graph anomalies, we propose a topology-aware generator G. Unlike Euclidean generators that focus solely on feature synthesis, G must simultaneously characterize the non-*i.i.d.* node distributions and the underlying network topology. Specifically, G establishes explicit links between synthetic nodes $\mathcal{V}_{\text{syn}}$ and real anomalies $\mathcal{V}_{\text{ab}}$, rather than the entire training set, to effectively reinforce the connectivity of the minority class. This targeted link generation also preserves graph homophily by avoiding noisy connections between synthetic anomalies and normal nodes. For computational efficiency, G is implemented as a three-layer fully connected network that maps Gaussian noise $\mathbf{Z} \in \mathbb{R}^{n_{\text{syn}} \times f}$ from the latent space Ω to the anomaly distribution. To rectify the data skewness, the number of synthetic nodes is determined by $n_{\text{syn}} = \alpha \times (n_{\text{nor}} - n_{\text{ab}})$, where $n_{\text{nor}} = |\mathcal{V}_{\text{nor}}|$ is the number of normal nodes and $n_{\text{ab}} = |\mathcal{V}_{\text{ab}}|$ is the number of anomalies. $\alpha \in (0, 1]$ serves as the generation ratio.

To synthesize informative anomalies, we propose a joint feature-and-structure generation strategy. First, the generator

maps latent noise $\mathbf{Z}$ to a topological space to establish links between synthetic nodes $\mathcal{V}_{\text{syn}}$ and real anomalies $\mathcal{V}_{\text{ab}}$:

$$\mathbf{S} = \mathbf{MLP}(\mathbf{Z}), \quad \tilde{\mathbf{S}} = \text{softmax}(\mathbf{S}). \tag{3}$$

where $\tilde{\mathbf{S}} \in \mathbb{R}^{n_{\text{syn}} \times n_{\text{ab}}}$ represents the normalized connectivity distribution. To derive a discrete topological structure, we apply a threshold η (typically set to $1/n_{\text{ab}}$), where $\tilde{\mathbf{S}}_{uv} = 1$ if $\tilde{\mathbf{S}}_{uv} \geq \eta$, and 0 otherwise.

To make the thresholding operation differentiable, we adopt the Straight-Through Estimator [41]. Specifically, we first apply a threshold function to the softmax-normalized connection matrix $\tilde{\mathbf{S}}$, yielding a binary connection matrix $\mathbf{M}$ whose entries are either 0 or 1. We then construct the final connection matrix as $\tilde{\mathbf{S}} + \text{sg}[\mathbf{M} - \tilde{\mathbf{S}}]$, where $\text{sg}[\cdot]$ denotes the stop-gradient operation.

The design in (3) is motivated by its capacity to transform Gaussian prior noise into a sparse affinity space that aligns with the inherent topological properties of graph data. By employing the subsequent thresholding operation, we facilitate the controllable synthesis of topological structures. This mechanism ensures that the generated connection patterns are analogous to the manifestation of real-world anomalies, which is crucial for capturing the fundamental structural characteristics of anomalous nodes in a realistic and robust manner.

Subsequently, synthetic features $\mathbf{X}_{\text{syn}}$ are generated by aggregating attributes from connected real anomalies to ensure feature-space alignment:

$$\mathbf{X}_{\text{syn}} = \tilde{\mathbf{S}}\mathbf{X}_{\text{ab}}, \tag{4}$$

where $\mathbf{X}_{\text{ab}} \in \mathbb{R}^{n_{\text{ab}} \times d}$ denotes the features of original anomalies.

By simultaneously synthesizing attributes and topological links, the generator yields a balanced attributed graph $\tilde{\mathcal{G}} = (\tilde{\mathcal{V}}, \tilde{\mathcal{E}}, \tilde{\mathbf{X}})$, where the newly generated components $(\mathcal{V}_{\text{syn}}, \mathcal{E}_{\text{syn}})$ are integrated with the original network to alleviate data scarcity and structural isolation.

B. Graph Neighbor Filter

The balanced graph $\tilde{\mathcal{G}}$ synthesized by G may still contain redundant connections that amplify the camouflage effect during message passing. To mitigate this, we propose the Graph Neighbor Filter (GNF), which adaptively prunes noisy edges between nodes with significant feature-space divergence. GNF iteratively refines the graph topology by evaluating the L_2 distance between node embeddings. Specifically, for each layer $k \in \{1, \ldots, K\}$, the process is formulated as:

$$\begin{aligned} &\mathbf{H}^{(k)} = \text{GCN}(\tilde{\mathbf{A}}^{(k-1)}, \mathbf{H}^{(k-1)}), \quad k = 1, 2, \ldots, K, \\ &\tilde{\mathbf{A}}_{uv}^{(k)} = \begin{cases} 0, & \text{if } ||\mathbf{h}_u{}^{(k)} - \mathbf{h}_v{}^{(k)}||_2 > \lambda, \\ \tilde{\mathbf{A}}_{uv}^{(k-1)} & \text{otherwise}, \end{cases} \end{aligned} \tag{5}$$

where $\tilde{\mathbf{A}}^{(k)}$ is the k-th adjacency matrix of the balanced graph after filtering out the irrelevant neighbors. λ is a distance threshold and $\mathbf{H}^{(0)} = \tilde{\mathbf{X}}$. At $k = 0$, the filtering is performed directly on the initial attributes $\tilde{\mathbf{X}}$ via the L_2 distance.

To ensure that the embeddings $\mathbf{H}^{(K)}$ are discriminative enough for effective filtering, the graph encoder is optimized via a cross-entropy loss:

$$\begin{aligned} &\hat{y}_v = \sigma\left(\mathbf{W}_g \mathbf{h}_v{}^{(K)}\right), \\ &\mathcal{L}_g = -\sum_{v \in \mathcal{V}_{\text{train}}} \left[y_v \log \hat{y}_v + (1 - y_v) \log(1 - \hat{y}_v)\right], \end{aligned} \tag{6}$$

where $\mathbf{W}_g$ and $\sigma(\cdot)$ are the learnable weight and the activation function, respectively. $\mathbf{h}_v{}^{(K)}$ is the embedding of node v at layer K, and $\hat{y}_v$ denotes the predicted probability for node v.

Upon completion of the iterative refinement, we obtain a rectified graph with reduced camouflage levels. Crucially, the rectified graph retains the original node attributes rather than the transformed embeddings. This DropEdge-like mechanism [42] introduces beneficial structural perturbations, which enhance the model's robustness and prevent the dilution of anomalous signals during subsequent representation learning.

C. Discriminator and Detector

To simultaneously ensure the fidelity and discriminability of the synthesized anomalies, we introduce a discriminator (D) and an auxiliary detector (T). Both modules adopt a GCN backbone and operate on the balanced graph $\tilde{\mathcal{G}}$, which incorporates the filtered topology $\tilde{\mathbf{A}}$ and the balanced attribute matrix $\tilde{\mathbf{X}}$. Specifically, the discriminator D is trained to distinguish between original nodes and those produced by the generator G. In contrast, the detector T is tasked with binary classification across the entire balanced node set, learning to differentiate normal nodes from both real and synthetic anomalies. By integrating these two objectives, the framework ensures that the generated nodes are not only realistic in the feature space but also effective in reinforcing the decision boundary for GAD.

D. Model Training

The training process of GFGAD is formulated as a triadic game among the generator G, the discriminator D, and the detector T. Building upon the foundational min-max objective of GANs [34], we define the specific loss functions for each component as follows.

Discriminator and Generator Objectives: The discriminator D is trained to maximize the probability of correctly assigning labels to both real and synthetic nodes, while G is optimized to minimize this probability by synthesizing high-fidelity anomalies.

The objective function for D is defined as:

$$\mathcal{L}_{\text{D}} = -\mathbb{E}_{v \in \mathcal{V}_{\text{train}}}[\log(\text{D}(v))] - \mathbb{E}_{z \in \Omega}[\log(1 - \text{D}(\text{G}(z)))], \tag{7}$$

where z is randomly sampled from a Gaussian noise space.

Conversely, the generator G is optimized to confuse D by minimizing the following adversarial loss:

$$\mathcal{L}_{\text{G}} = \mathbb{E}_{z \in \Omega}[\log(1 - \text{D}(\text{G}(z)))]. \tag{8}$$

Through this minimax competition, G learns to project latent noise z into a distribution that closely approximates the feature and structural properties of real data.

Detector Objective: While D focuses on sample fidelity, the detector T is optimized to differentiate between normal and

Algorithm 1: GFGAD Training.

Input: $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$, Number of training epochs N, Number of steps l for D.

Initialize the weight parameters.
for epoch $\leftarrow 1$ to N **do**
  Generate the balanced graph $\tilde{\mathcal{G}} = (\tilde{\mathcal{V}}, \tilde{\mathcal{E}}, \tilde{\mathbf{X}})$ using (3) and (4).
  Apply GNF to derive rectified graph $\tilde{\mathcal{G}}$ using (5).
  Input $\tilde{\mathcal{G}}$ into D and T
  for step $\leftarrow 1$ to l **do**
    Compute the node representations of $\mathcal{V}_{\text{train}}$ and $\mathcal{V}_{\text{syn}}$ using (2).
    Update the discriminator D using (7).
  end for
  Update the generator G using (8).
  Update the GNF using via $\mathcal{L}_g$ (6).
  Obtain node features of $\mathcal{V}_{\text{ab}}$, $\mathcal{V}_{\text{syn}}$ and $\mathcal{V}_{\text{nor}}$ via (2).
  Update the detector T using (9).
end for

Output: Trained detector T and inference results.

abnormal distributions across the balanced graph. Let $\mathcal{V}_{\text{syn}}$ denote the set of nodes synthesized by G. The detection loss is formulated as the cross-entropy objective:

$$\begin{aligned}\mathcal{L}_{\text{T}} = & -\mathbb{E}_{v\in\mathcal{V}_{\text{ab}}}[\log(\text{T}(v))] - \mathbb{E}_{v\in\mathcal{V}_{\text{syn}}}[\log(\text{T}(v))] \\ & -\mathbb{E}_{v\in\mathcal{V}_{\text{nor}}}[\log(1-\text{T}(v))]. \end{aligned} \tag{9}$$

This objective ensures that the generated anomalies are not only realistic but also strategically positioned to reinforce the discriminative boundary between normal nodes and outliers.

Joint Optimization: Since all nodes in $\mathcal{V}_{\text{syn}}$ are assigned anomalous labels, the detector T serves to establish a robust boundary between the normal and the augmented anomalous distributions (comprising both $\mathcal{V}_{\text{ab}}$ and $\mathcal{V}_{\text{syn}}$). Crucially, as $\mathcal{V}_{\text{syn}}$ is a direct output of the generator, the optimization of $\mathcal{L}_{\text{T}}$ in (9) simultaneously refines G. This establishes a cooperative learning paradigm where G is incentivized to synthesize anomalies that align with the discriminative patterns captured by T. Upon convergence, T attains a superior capability to generalize across diverse anomaly patterns, facilitating effective inference on the rectified graph.

Furthermore, the integration of both D and T serves as a strategic defense against mode collapse, a common instability in standard GAN architectures [43]. In GFGAD, the generator G must concurrently satisfy the authenticity constraints of D and the discriminative requirements of T. This dual-objective pressure compels G to synthesize more diverse and representative samples.

The detailed training procedure of GFGAD is summarized in Algorithm 1. The overall framework is optimized in an iterative manner. In each epoch, we first update the GNF to refine the topology, followed by the adversarial updates of G, D, and T. This collaborative training ensures the robustness of the final detection mapping $\mathcal{H}$.

TABLE II
STATISTICS OF THE DATASETS

Dataset	#Nodes	#Edges	#Features	#AR
Amazon	11,944	4,398,392	25	6.87%
Enron	13,533	176,987	18	4.47%
ACM	16,484	71,980	8,337	3.64%
Flickr	7,575	239,738	12,047	5.94%
BlogCatalog	5,196	171,743	8,189	5.77%

As a theoretical foundation, we derive the generalization error bounds for the generator, discriminator, and detector in the Appendix, providing a rigorous characterization of the model's generalization capability.

V. Complexity Analysis

In this section, we evaluate the computational efficiency of GFGAD by analyzing its time complexity. The overall complexity is primarily governed by the adversarial training process. Specifically, the generator G leverages MLP to map latent noise into topological and feature spaces; the complexity for constructing the normalized connectivity distribution $\tilde{\mathbf{S}}$ is $\mathcal{O}(n_{\text{syn}}n_{\text{ab}}f)$, while the subsequent feature synthesis via matrix multiplication incurs $\mathcal{O}(n_{\text{syn}}n_{\text{ab}}d)$. Consequently, the total complexity for the generation stage is $\mathcal{O}(n_{\text{syn}}n_{\text{ab}}(f+d))$, where n_{syn} and n_{ab} denote the number of synthesized and real anomalies, respectively, while f and d represent the dimensions of the Gaussian noise and node features. Simultaneously, the discriminator D and detector T, both implemented with GCN architectures, operate on a balanced graph of $(n + n_{\text{syn}})$ nodes with an adjacency matrix $\tilde{\mathbf{A}}$. The complexity per GCN layer is $\mathcal{O}(\|\tilde{\mathbf{A}}\|_0 d + (n+n_{\text{syn}})d^2)$ [44], where $\|\tilde{\mathbf{A}}\|_0$ characterizes the structural sparsity of the graph. Combining these components, the total time complexity per training epoch is approximately $\mathcal{O}(n_{\text{syn}}n_{\text{ab}}(f+d) + \|\tilde{\mathbf{A}}\|_0 d + (n+n_{\text{syn}})d^2)$. Given that $\|\tilde{\mathbf{A}}\|_0$ typically scales linearly with the number of nodes in sparse real-world networks and n_{syn} is proportional to the minority class size, this complexity ensures that GFGAD maintains computational feasibility for large-scale GAD tasks.

VI. Experiments

A. Datasets

To evaluate the effectiveness of the proposed GFGAD model, we employ five widely recognized benchmark datasets for GAD: **Amazon** [45], **Enron** [46], **ACM** [47], **Flickr** [48], and **BlogCatalog** (abbreviated as **Blog**) [49]. The statistical properties of these datasets are summarized in Table II, where #Nodes and #Edges denote the total counts of nodes and edges, respectively; #Features indicates the dimensionality of node attributes; and #AR represents the anomaly ratio. The specific characteristics of each dataset are detailed below:

- *Amazon [45]:* A product review dataset specifically covering the musical instrument category. Nodes represent users, and edges denote shared review behaviors (e.g., reviewing at least one identical product). The objective is to identify fraudulent users who post malicious reviews.

- *Enron [46]:* A corporate email communication network where nodes represent employees and edges indicate email exchanges. In this context, the task is to detect spammers or anomalous communication patterns.
- *ACM [47]:* An academic citation network in which nodes correspond to scientific papers and edges represent citation relationships. This dataset is primarily used for identifying anomalous citation behaviors.
- *Flickr [48]:* A social network derived from the image-sharing platform, where nodes represent users and edges denote "following" relationships.
- *BlogCatalog [49]:* A social graph collected from the BlogCatalog website, where nodes represent bloggers and edges reflect mutual social connections. For both Flickr and BlogCatalog, the goal is to detect malicious users who generate abnormal follow-ship patterns.

Among the selected benchmarks, Amazon is inherently a heterogeneous graph, while the remaining four are homogeneous. To ensure compatibility with the GCN-based architecture, we follow established preprocessing protocols by utilizing the "homo" edges of the Amazon dataset to treat it as a homogeneous graph. Furthermore, as the ACM, Flickr, and BlogCatalog datasets do not contain ground-truth anomalies, we adopt the widely used synthetic injection method [50], [51] to generate both structural and attribute anomalies. Specifically, structural anomalies are simulated by creating cliques of densely connected nodes, where we randomly select a subset of m nodes and inject $m/2$ random edges for each node within this subset to ensure high interconnectivity. For attribute anomaly injection, we randomly select n nodes as candidates; for each candidate node i, we identify a reference set of s nodes and locate the node j that exhibits the maximum attribute divergence (i.e., the largest L_2 distance) from i, subsequently replacing the attributes of node i with those of node j. This injection process is repeated N times to construct the complete anomalous set. In our experiments, we set $m = n = 15$ and $s = 50$, with N configured as 10, 15, and 20 for BlogCatalog, Flickr, and ACM, respectively. Consistent with the evaluation protocols of previous models [6], [15], we utilize 40% of the data for training to ensure a fair and rigorous comparison.

B. Baselines

To comprehensively evaluate the performance of GFGAD, we compare it against three main categories of competitive baselines: (I) *Classic GNNs*, (II) *GAN-based models*, and (III) *specialized GNN-based GAD frameworks*. The specialized GAD models are further categorized into *data augmentation-based*, *filtering-based*, and *spectral-based approaches*.

(I) *Classic GNNs:*
- *GCN [4]:* A foundational model that integrates graph topology and node features via a first-order polynomial approximation of spectral filters.
- *DR-GCN [52]:* A dual-regularized GCN that utilizes conditional adversarial training and distribution alignment to learn robust node representations.

(II) *GAN-based Models:*
- *GAAN [39]:* An architecture inspired by Efficient-GAN [53] that incorporates both attribute and structural information within its encoding module.
- *ImGAGN [54]:* A generative adversarial graph network designed to synthesize minority nodes, thereby simulating the distribution of the minority class to alleviate imbalance.

(III) *GNN-based GAD Models:*
- *Data Augmentation-based Models:* These models enhance the minority class to improve discriminative power. **CONSISGAD** [31] employs a learnable augmentation mechanism for consistency training under limited supervision, while **SpaceGNN** [55] utilizes a spatial projection function and weighted homogeneity to model distance-aware relationships in an ensemble space.
- *Filtering-based Models:* These methods explicitly or implicitly mitigate redundant or noisy connections. **PC-GNN** [27] uses a label-balanced sampler to address class imbalance during subgraph training. **SparseGAD** [29] leverages graph sparsification and homophily-heterophily analysis to reduce structural noise. **GHRN** [15] prunes inter-class edges in the spectral domain to partition high-frequency components, and **DR-GGAD** [56] introduces a multi-scale hyper-residual center and affinity residual modules to measure node-to-center distances.
- *Spectral-based Models:* These frameworks exploit various spectral properties of graphs. **AMNet** [28] adaptively combines high- and low-frequency signals. **BWGNN** [6] utilizes beta kernel functions and bandpass filters to capture localized spectral anomalies. **PMP** [30] implements partitioned message passing to adaptively aggregate information from heterophilic and homophilic neighbors. **APF** [57] identifies node-specific subgraphs via the Rayleigh quotient and employs learnable polynomial filters. Finally, **SAGAD** [58] leverages reparameterized Chebyshev filters to capture multi-hop, frequency-aware patterns.

C. Evaluation Metrics

Following standard practices in GAD, we employ Area Under the Receiver Operating Characteristic Curve (**AUC-ROC**) and Area Under the Precision-Recall Curve (**AUC-PR**) as the primary evaluation metrics. While AUC-ROC provides an aggregate measure of the model's discriminative capability, AUC-PR offers a more rigorous assessment in the presence of highly skewed class distributions. For both metrics, higher values signify superior detection performance.

D. Implementation Details

We implement GFGAD using PyTorch and execute all experiments on a workstation equipped with an NVIDIA GeForce RTX

TABLE III
EVALUATION PERFORMANCE (%) OF DIFFERENT MODELS ON FOUR EVALUATED DATASETS. ROC: AUC-ROC. PR: AUC-PR

Dataset	Amazon		Enron		ACM		Flickr		Blog	
	ROC	PR	ROC	PR	ROC	PR	ROC	PR	ROC	PR
GCN	74.34	25.66	78.12	18.66	72.06	23.15	72.75	29.32	68.83	15.44
DR-GCN	74.91	23.55	78.46	19.32	72.37	22.47	71.62	28.57	65.19	16.12
ImGAGN	82.01	35.12	80.05	19.26	75.47	33.45	72.68	31.60	73.20	29.21
GAAN	79.21	33.03	77.52	18.32	74.90	30.12	71.43	32.59	73.91	30.60
CONSISGAD	97.50	88.19	92.95	64.75	79.26	36.66	72.29	33.15	72.29	30.22
SpaceGNN	96.63	86.46	93.64	62.24	79.43	37.56	73.19	33.76	74.25	22.51
PCGNN	92.61	81.53	87.86	65.24	77.94	35.12	74.00	29.12	72.22	29.12
SparseGAD	95.78	87.18	95.43	66.22	70.84	34.38	72.70	22.47	68.03	18.64
GHRN	**97.68**	**88.21**	93.79	66.02	72.88	25.73	72.32	33.39	74.01	31.99
DR-GGAD	94.16	85.25	87.67	65.28	75.16	35.88	73.90	32.59	75.54	31.95
AMNet	92.85	80.51	93.06	65.92	73.95	31.55	72.15	31.56	69.52	25.62
BWGNN	96.55	87.58	91.36	65.79	72.72	23.99	65.89	32.46	70.10	31.90
PMP	97.57	88.16	92.29	65.96	80.18	37.47	72.50	34.22	73.03	30.11
APF	93.59	86.76	93.41	65.95	62.56	42.91	71.90	33.86	75.97	30.47
SAGAD	96.62	85.14	94.40	66.08	61.87	35.99	71.65	34.45	68.27	31.39
$\text{GFGAD}_{\text{smote}}$	89.68	80.15	89.52	62.32	75.80	33.37	68.49	30.54	74.83	29.98
GFGAD_{T}	91.61	83.54	92.33	63.19	79.74	36.59	72.72	32.86	75.22	30.24
GFGAD_{D}	92.27	83.97	93.18	63.59	80.27	37.97	73.35	33.29	75.88	30.65
$\text{GFGAD}_{\text{GNF}}$	93.78	84.26	93.69	65.20	80.32	39.02	73.72	34.47	76.82	31.36
GFGAD	95.65	86.35	**95.63**	**67.35**	**81.62**	**40.96**	**74.62**	**35.22**	**77.52**	**32.75**

3090 GPU. The graph encoder, discriminator, and detector all adopt a 2-layer GCN architecture (i.e., $k = 2$ in (2)) with 128-dimensional hidden units and ReLU activation functions. Notably, these modules maintain independent, non-shared weight parameters. Regarding hyperparameter configurations, the generation ratio α and the distance threshold λ are empirically set to 1 and 0.5, respectively. We utilize the Adam optimizer with a learning rate of 0.001. The maximum number of training epochs N and the discriminator steps l are both fixed at 50. For all baseline models, we utilize the official source codes provided by the respective authors and adhere to the hyperparameter settings recommended in their original publications.

VII. RESULTS

A. Performance Comparison

To mitigate the impact of random initialization, all models are executed ten times, with the mean AUC-ROC and AUC-PR values reported in Table III. Based on these quantitative results, we provide the following in-depth analysis:

- *Overall Effectiveness:* GFGAD consistently achieves superior performance across various datasets, validating its efficacy in GAD tasks. This superiority stems from our departure from conventional GNN-based representation learning; instead, GFGAD constructs a balanced, rectified graph. Specifically, the GAN-based architecture synthesizes informative anomalies with consistent attributes and structural dependencies, while the graph neighbor filter yields a subgraph with enhanced homophily. By synergistically combining generation and filtration, the detector effectively captures the latent patterns of both normal and anomalous nodes.
  Moreover, as shown in Table III, on the Enron dataset—which contains naturally occurring, real-world anomalies—our method consistently and substantially outperforms all competing baselines. On the Amazon dataset, our method also achieves highly competitive performance, ranking among the top-performing approaches. These results confirm that GFGAD's effectiveness is not an artifact of the injection strategy, but rather stems from its intrinsic capability to capture structural and attributive anomaly patterns in real-world graphs.
- *Comparison with Specialized GAD Baselines:* We compare GFGAD against GAD models across three categories: data augmentation-based (e.g., CONSISGAD), filtering-based (e.g., GHRN), and spectral-based (e.g., SAGAD) methods. Although these specialized models generally outperform traditional GNNs, they remain significantly inferior to GFGAD in most datasets. We attribute this performance gap to the fact that existing methods typically rely on a single strategy to address the camouflage issue. In contrast, GFGAD employs a dual-strategy-adversarial augmentation and redundant connection filtration to construct a refined, balanced graph, enabling more precise class separation.
- *Comparison with GAN-based Models:* GFGAD exhibits a decisive advantage over standard GAN architectures.

For instance, compared to ImGAGN, our model achieves a 13.64% improvement in AUC-ROC and a remarkable 51.23% increase in AUC-PR on the Amazon dataset. This significant margin highlights the importance of our auxiliary detector and graph neighbor filter, which prioritize the generation of high-quality, task-relevant anomalies. In contrast, standard GANs often focus on synthesizing realistic normal instances, which contributes little to mining the discriminative features necessary for GAD.

- *Comparison with Classic GNNs:* Classic GNNs (e.g., GCN) typically treat GAD as a vanilla binary classification task, neglecting the inherent challenges of overfitting and camouflage. GFGAD markedly outperforms these models; for example, it surpasses the best-performing GCN by 1.87% in AUC-ROC and 5.90% in AUC-PR on the Flickr dataset. The substantial performance disparity underscores that simple GNN architectures are insufficient for resolving the complex structural and distributional imbalances present in GAD.

B. Ablation Study

To investigate the contribution of each core component in GFGAD, we design four model variants and conduct a series of ablation studies. The variants are defined as follows:

(I) *$GFGAD_{smote}$:* The GAN-based synthesis strategy is replaced with a simple interpolation-based method similar to SMOTE [59]. The topology of generated nodes is determined by the inner product of their features.

(II) *$GFGAD_{T}$:* The auxiliary detector T is removed, and a standard MLP classifier is employed to categorize the balanced data.

(III) *$GFGAD_{D}$:* The discriminator D is removed, leaving the generator G and detector T to interact in a simplified cooperative framework.

(IV) *$GFGAD_{GNF}$:* The graph neighbor filter is disabled, and the model operates directly on the raw structure of the balanced attributed graph.

Based on the results presented in Table III, we draw the following conclusions. *First*, each component is indispensable, as the removal of any module leads to a significant performance degradation. Specifically, $\text{GFGAD}_{\text{smote}}$ exhibits a substantial drop in efficacy, primarily because fixed feature interpolation lacks the optimization feedback necessary to synthesize high-quality, task-relevant anomalies. In contrast, our adversarial generation mechanism adaptively explores the latent space to produce more informative samples.

Second, both the discriminator D and the detector T are vital for stable adversarial training. D ensures that synthetic nodes remain distributionally consistent with real nodes, preventing "messy" or unrealistic generations, while T provides implicit guidance to G to facilitate the production of nodes with discriminative anomalous patterns.

Third, the introduction of the graph neighbor filter effectively mitigates the contamination caused by redundant connections (camouflage). To further validate its efficacy, we report the percentages of filtered redundant links in Table IV. The empirical results demonstrate that our filter successfully prunes connections between normal and anomalous nodes.

TABLE IV
PERCENTAGES OF REDUNDANT CONNECTIONS FILTERED IN DIFFERENT LAYERS OF THE GNN MODEL ON DIFFERENT DATASETS

Dataset	Amazon	Enron	ACM	Flickr	Blog
1-layer	15.69%	20.52%	19.24%	15.54%	13.83%
2-layer	22.66%	25.39%	30.26%	25.22%	36.52%

Furthermore, we compare our filter with a random perturbation strategy (Ours-rand), which randomly adds edges between anomalies and removes edges between normal-anomalous pairs within the training set. As shown in Table V, GFGAD significantly outperforms Ours-rand. This is because our graph neighbor filter performs edge pruning at a global level based on learned homophily, whereas random strategies are restricted to the training set and lack the structural awareness required for effective denoising.

C. Visualization

To qualitatively evaluate the discriminative power of GFGAD, we utilize the t-SNE [60] algorithm to project the learned high-dimensional node embeddings into a two-dimensional space. We select three representative baselines (i.e., GAAN, SparseGAD, and DR-GCN) for a comparative visualization analysis on the BlogCatalog dataset, as illustrated in Fig. 3.

Our observations are as follows. *First*, as shown in Fig. 3(a), the synthetic nodes generated by GFGAD are distributed in close proximity to the ground-truth anomalies. This proximity indicates that our GAN-based generation strategy successfully captures and simulates the underlying patterns of real-world anomalies, providing effective supervision for the detector. *Second*, a clear decision boundary is observed in GFGAD's embedding space, where normal and anomalous nodes (including both real and synthetic instances) form distinct, well-separated clusters. This superior separation can be attributed to GFGAD's ability to leverage the balanced, rectified graph to learn more robust and discriminative representations.

In contrast, as depicted in Fig. 3(b)–(d), the baseline models struggle to effectively partition the two classes. The embeddings of normal and anomalous nodes appear intertwined or overlapping to varying degrees. This phenomenon suggests that without explicit handling of class imbalance and structural camouflage, existing models are prone to overfitting or noise interference, ultimately failing to map different classes into linearly separable regions in the latent space.

D. Sensitivity Analysis

We conduct a series of empirical studies to evaluate the sensitivity of GFGAD with respect to four key hyperparameters: the generation ratio $\alpha \in [0.1, 1]$, the distance threshold $\lambda \in [0.1, 1]$, the topology threshold $\eta \in [\frac{1}{3n_{\text{ab}}}, \frac{3}{n_{\text{ab}}}]$, and the number of graph

TABLE V
MODEL PERFORMANCE ON FOUR DATASETS WITH DIFFERENT STRATEGIES. ROC: AUC-ROC. PR: AUC-ROC

Dataset	Amazon		Enron		ACM		Flickr		Blog	
	ROC	PR	ROC	PR	ROC	PR	ROC	PR	ROC	PR
Ours-rand	93.16	84.11	93.26	65.71	74.46	32.72	71.29	31.16	73.95	27.40
Ours	95.65	86.35	95.63	67.35	81.62	40.96	74.62	35.22	77.52	32.75

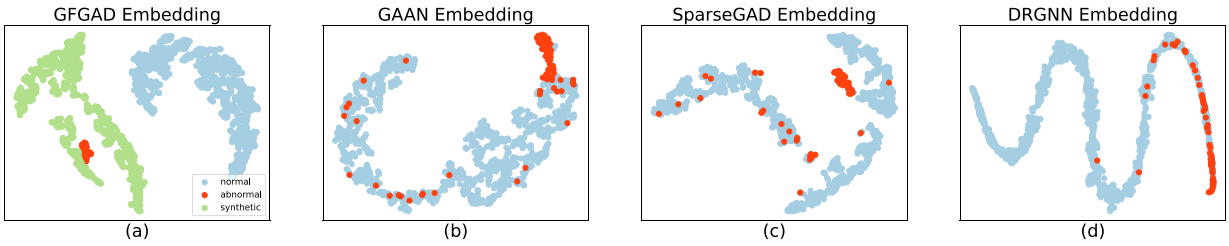


Fig. 3. Visualization of different methods on the BlogCatalog dataset.

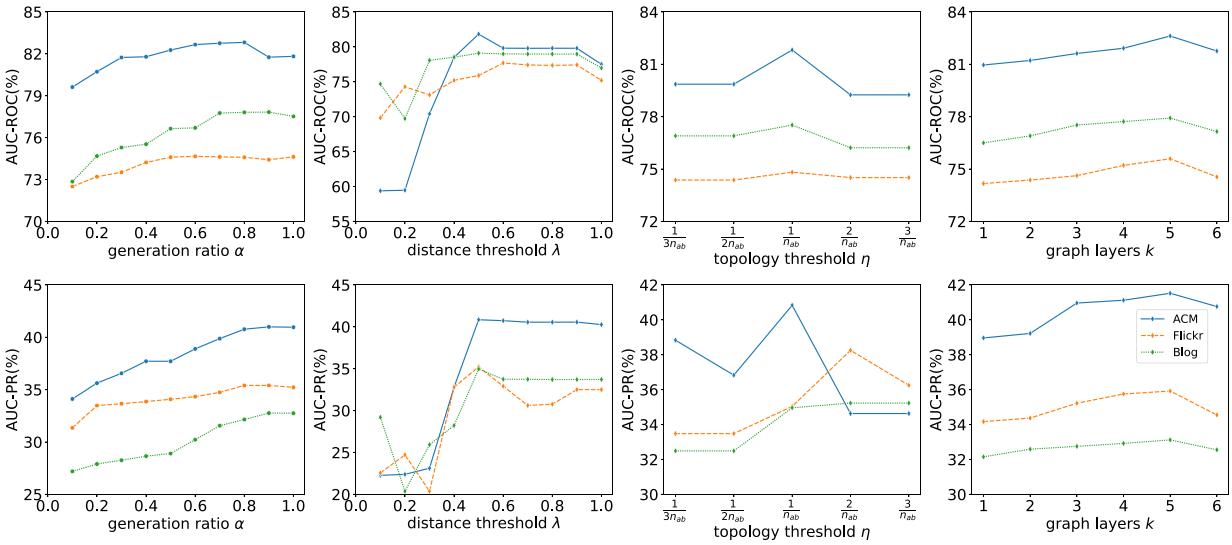


Fig. 4. Evaluation performance achieved in terms of relevant metrics with different hyperparameters on several datasets. The first line reports AUC-ROC results, and the second line reports the AUC-PR results.

TABLE VI
STATISTICS OF LARGE DATASETS

Dataset	# Nodes	# Edges	# Features	# AR
ogbn-arxiv	169,343	1,166,243	128	3.50%
Flickr-Full	89,250	899,756	500	2.58%

layers k ranging from 1 to 6 in the neighbor filter. The experimental results across three representative datasets (ACM, Flickr, and BlogCatalog) are illustrated in Fig. 4 and analyzed as follows:

(I) *Impact of generation ratio α:* As α increases, both AUC-ROC and AUC-PR metrics exhibit an initial upward trend followed by minor fluctuations within a stable range. This suggests that the synthesized high-quality anomalies effectively facilitate the extraction of discriminative features. However, performance plateaus once a certain threshold is reached, indicating that $\alpha \in [0.5, 1]$ is generally sufficient for balancing the data distribution. Notably, GFGAD maintains competitive performance even at low generation ratios, demonstrating its robustness to the volume of synthetic supervision.

(II) *Impact of distance threshold λ:* The model performance is suboptimal when λ is too small, as it tends to over-prune valid connections between normal nodes, thereby destroying essential structural information. Conversely, an excessively large λ fails to filter sufficient noisy connections, allowing structural camouflage to persist. Our empirical observations suggest that a range of $\lambda \in [0.5, 0.8]$ typically yields the most favorable trade-off between noise reduction and structural preservation.

(III) *Impact of topology threshold η:* Optimal results are consistently achieved when η is near $1/n_{\text{ab}}$ for all datasets. Deviations from this value—either too high or too low—lead to either overly sparse or excessively redundant connections between synthetic and real anomalies. Such structural misconfigurations impede the model's ability

TABLE VII
EVALUATION PERFORMANCE (%) ACHIEVED ON TWO LARGER DATASETS WITH DIFFERENT PROPORTIONS OF TRAINING DATA. ROC: AUC-ROC. PR: AUC-PR

Dataset	Metric	CONSISGAD	SpaceGNN	SparseGAD	GHRN	DR-GGAD	AMNet	BWGNN	PMP	APF	SAGAD	Ours
ogbn-arxiv	ROC	93.15	93.96	82.55	92.80	90.12	71.65	91.93	93.29	92.19	93.10	**95.83**
	PR	37.96	42.56	29.29	33.60	37.15	23.12	33.60	35.99	37.56	41.15	**55.92**
Flickr-Full	ROC	84.56	84.25	79.25	83.57	82.19	67.42	85.11	84.19	85.01	83.79	**86.16**
	PR	26.32	32.79	20.96	22.79	29.12	5.28	21.12	25.29	32.12	37.52	**43.32**

TABLE VIII
RESULTS OF OUR MODEL BY USING ALTERNATIVE GNN ARCHITECTURES. ROC: AUC-ROC. PR: AUC-PR

Dataset	Amazon		Enron		ACM		Flickr		BlogCatalog	
	ROC	PR	ROC	PR	ROC	PR	ROC	PR	ROC	PR
GFGAD-SGC	84.01	69.03	83.59	55.06	72.61	32.54	68.56	30.56	64.53	28.98
GFGAD-SAGE	95.09	85.21	94.98	64.92	81.34	39.07	73.15	42.72	75.23	31.80
GFGAD-GCN	**95.65**	**86.35**	**95.63**	**67.35**	**81.62**	**40.96**	**74.62**	**35.22**	**77.52**	**32.75**

TABLE IX
COMPUTATIONAL COSTS OF DIFFERENT COMPONENTS ON THE EVALUATED DATASETS, INCLUDING FLOPS AND THE NUMBER OF PARAMETERS

Datasets	G		D (T)	
	FLOPs	Params	FLOPs	Params
Amazon	1,030M	0.19M	1.89M	0.003M
Enron	230M	0.02M	2.42M	0.002M
ACM	1,710M	0.14M	2.89M	1.07M
Flickr	589M	0.11M	1.31M	1.54M
Blog	295M	0.08M	0.90M	1.05M
ogbn-arxiv	81,060M	0.63M	30.04M	0.02M
Flickr-Full	41,310M	0.62M	15.69M	0.06M

TABLE X
COMPUTATIONAL COSTS OF SPACEGNN ON THE EVALUATED DATASETS, INCLUDING FLOPS AND THE NUMBER OF PARAMETERS

Datasets	SpaceGNN	
	FLOPs	Params
Amazon	5,298,491M	1.08M
Enron	42,099M	0.36M
ACM	1,056,184M	10.65M
Flickr	739,829M	14.93M
Blog	344,441M	10.49M
ogbn-arxiv	1,120,598M	1.20M
Flickr-Full	1,270,578M	1.63M

TABLE XI
COMPARISON OF RUNTIME (SECOND) FOR DIFFERENT MODELS

Dataset	SpaceGNN	DG-GGAD	APF	SAGAD	Ours
Amazon	171.47	27.79	15.92	19.20	15.20
Enron	50.93	12.32	10.36	17.52	10.21
ACM	157.30	15.95	20.99	26.75	17.81
Flickr	175.17	5.19	7.25	10.95	5.53
Blog	133.22	4.79	10.29	12.32	5.85

to learn consistent anomalous patterns, thereby impairing its overall recognition capability.

(IV) *Impact of graph layers k:* The model performance follows a bell-shaped curve as k increases, typically peaking at $k = 5$. A plausible explanation is that deeper layers in our filtering mechanism enable more thorough elimination of redundant edges. It is noteworthy that while standard GCNs often suffer from over-smoothing beyond $k = 2$, GFGAD maintains high performance at greater depths. This highlights the effectiveness of our filtering strategy, which functions as a task-specific edge-pruning mechanism to mitigate the over-smoothing issue common in deep GNNs.

E. Performance on Large Datasets

To further evaluate the scalability and robustness of GFGAD, we conduct a comparative analysis on two large-scale benchmark datasets: **ogbn-arxiv** [61] and **Flickr-Full** [9]. The statistical properties of these datasets are summarized in Table VI, and their specific characteristics are detailed below:

- *ogbn-arxiv [61]:* A comprehensive citation network among computer science papers. Nodes represent individual papers, while edges denote citation relationships. Node features are 128-dimensional vectors derived from the mean word embeddings of each paper's title and abstract.
- *Flickr-Full [9]:* A more expansive version of the Flickr dataset, where nodes correspond to images and edges represent shared attribute similarities. Node attributes are represented by 500-dimensional bag-of-words vectors extracted from the images.

The anomaly injection protocols for these datasets remain consistent with the methodology described in Section VI-A. As reported in Table VII, GFGAD consistently achieves the state-of-the-art performance across all evaluation metrics, maintaining a significant lead over the baselines even as the graph scale increases. For instance, on the ogbn-arxiv dataset, GFGAD outperforms the second-best model by a substantial margin of 13.36% in terms of AUC-PR. These results demonstrate that our "Generate and Filter" strategy is not only effective on standard benchmarks but also scales efficiently to larger, more complex graph structures.

F. Flexibility Study

To validate the architectural flexibility of GFGAD, we evaluate its performance by substituting the default GCN backbone in the discriminator D and detector T with other representative GNN architectures, specifically GraphSAGE [62] and SGC [24]. As summarized in Table VIII, our framework consistently achieves superior performance when integrated with both GCN and GraphSAGE backbones across all datasets. This phenomenon demonstrates that by effectively addressing the fundamental challenges of class imbalance and structural

camouflage, GFGAD can yield promising outcomes without necessitating overly sophisticated or task-specific backbone architectures. Such adaptability highlights the potential of our "Generate and Filter" strategy to be seamlessly integrated into diverse graph learning pipelines. However, we observe a performance degradation when utilizing the SGC backbone. This is likely due to SGC's inherent limitation as a linearized model, which may fail to capture the high-order non-linear patterns essential for distinguishing subtle anomalies from normal nodes.

G. Computational Complexity

To assess the practical feasibility of GFGAD, we analyze its computational overhead in terms of both model capacity and inference efficiency. Table IX details the floating-point operations (FLOPs) and parameter counts for the constituent modules (G, D, and T). For a comparative benchmark, we also report the complexity metrics of SpaceGNN in Table X. The empirical results demonstrate that GFGAD maintains a high degree of efficiency; across all evaluated datasets, it consistently requires fewer parameters and lower FLOPs than SpaceGNN. This efficiency suggests that GFGAD effectively balances detection performance with computational economy. Furthermore, despite the adversarial training architecture, the model exhibits a manageable memory footprint, ensuring that both training and inference are well-within the VRAM capacity of standard professional GPUs. These results underscore GFGAD's suitability for real-world applications where low latency and resource efficiency are paramount.

To empirically validate the scalability of GFGAD, we have supplemented the manuscript with a comparison of the training time per epoch between GFGAD and recent methods (SpaceGNN, DG-GGAD, APF, and SAGAD) on the evaluation datasets. The results are presented in Table XI. Despite our framework involving an additional generator, the per-epoch training time of GFGAD is comparable to or even lower than that of other GNN-based anomaly detectors, confirming that our sparsity design effectively controls the message-passing overhead in practice.

VIII. Conclusion and Future Work

In this paper, we introduced GFGAD, a novel "Generate and Filter" framework designed to address the inherent feature camouflage and structural sparsity challenges in GAD. By synergistically integrating a generative adversarial mechanism with an auxiliary detector, our model synthesizes high-quality, informative anomalies that provide robust supervision for representation learning. Furthermore, the introduction of a graph neighbor filter effectively mitigates structural noise by pruning redundant connections, thereby enhancing the model's ability to capture essential anomalous patterns. Extensive experiments on seven real-world datasets, including large-scale benchmarks, demonstrate that GFGAD consistently outperforms competitive baselines across multiple evaluation metrics.

While GFGAD establishes a high benchmark for GAD, it also opens several avenues for future research. One promising direction is the optimization of generative efficiency to better support industrial-scale graphs with hundreds of millions of nodes. Additionally, a key limitation of our current framework is the reliance on the pre-defined generation ratio α, which controls the number of synthetic anomalies generated per training iteration. While this parameter can be reasonably set based on validation performance or domain knowledge, its optimal value may vary across different real-world scenarios where the underlying anomaly ratio is unknown. We aim to investigate adaptive mechanisms for automatically determining the optimal generation ratio α in fully unsupervised or open-world scenarios. Finally, incorporating automated machine learning (AutoML) or self-supervised heuristics to refine the hyperparameter space could further enhance the model's robustness and ease of deployment in diverse practical applications.

References

[1] M. Kloft and P. Laskov, "Security analysis of online centroid anomaly detection," *J. Mach. Learn. Res.*, vol. 13, no. 1, pp. 3681–3724, 2012.

[2] Z. Liu et al., "GeniePath: Graph neural networks with adaptive receptive paths," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4424–4431.

[3] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, "Enhancing graph neural network-based fraud detectors against camouflaged fraudsters," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2020, pp. 315–324.

[4] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–14.

[5] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–12.

[6] J. Tang, J. Li, Z. Gao, and J. Li, "Rethinking graph neural networks for anomaly detection," in *Proc. Int. Conf. Mach. Learn.*, 2022, vol. 162, pp. 21076–21089.

[7] T. Zhao, C. Deng, K. Yu, T. Jiang, D. Wang, and M. Jiang, "Error-bounded graph anomaly loss for GNNs," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2020, pp. 1873–1882.

[8] J. Wang, R. Wen, C. Wu, Y. Huang, and J. Xion, "FdGars: Fraudster detection via graph convolutional networks in online app review system," in *Proc. Companion Proc. World Wide Web Conf.*, 2019, pp. 310–316.

[9] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. K. Prasanna, "Graphsaint: Graph sampling based inductive learning method," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–19.

[10] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.

[11] X. Ma et al., "A comprehensive survey on graph anomaly detection with deep learning," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12012–12038, Dec. 2023.

[12] F. Shi, Y. Cao, Y. Shang, Y. Zhou, C. Zhou, and J. Wu, "H2-FDetector: A GNN-based fraud detector with homophilic and heterophilic connections," in *Proc. ACM Web Conf.*, 2022, pp. 1486–1494.

[13] J. Ren, F. Xia, I. Lee, A. N. Hoshyar, and C. C. Aggarwal, "Graph learning for anomaly analytics: Algorithms, applications, and challenges," *ACM Trans. Intell. Syst. Technol.*, vol. 14, no. 2, pp. 28:1–28:29, 2022.

[14] H. Zheng et al., "Smoke screener or straight shooter: Detecting elite sybil attacks in user-review social networks," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–17.

[15] Y. Gao, X. Wang, X. He, Z. Liu, H. Feng, and Y. Zhang, "Addressing heterophily in graph anomaly detection: A perspective of graph spectrum," in *Proc. ACM Web Conf.*, 2023, pp. 1528–1538.

[16] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.

[17] R. Guan, Y. Liu, X. Feng, and X. Li, "Paper-publication prediction with graph neural networks," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 617–626.

[18] Y. Liu, M. Li, X. Li, F. Giunchiglia, X. Feng, and R. Guan, "Few-shot node classification on attributed networks with graph meta-learning," in *Proc. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2022, pp. 471–481.

[19] L. Yao, C. Mao, and Y. Luo, "Graph convolutional networks for text classification," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 7370–7377.

[20] Y. Liu, R. Guan, F. Giunchiglia, Y. Liang, and X. Feng, "Deep attention diffusion graph neural networks for text classification," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2021, pp. 8142–8152.
[21] Y. Liu et al., "Local and global: Temporal question answering via information fusion," in *Proc. Int. Joint Conf. Artif. Intell.*, 2023, pp. 5141–5149.
[22] Y. Liu, D. Liang, F. Fang, S. Wang, W. Wu, and R. Jiang, "Time-aware multiway adaptive fusion network for temporal knowledge graph question answering," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, 2023, pp. 1–5.
[23] G. Yang, M. Li, H. Feng, and X. Zhuang, "Deeper insights into deep graph convolutional networks: Stability and generalization," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 48, no. 2, pp. 1707–1719, Feb. 2026.
[24] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6861–6871.
[25] Y. Xie, J. Jia, C. Wen, D. Li, and M. Li, "Multi-topology contrastive graph representation learning," *Sci. China Inf. Sci.*, vol. 69, no. 2, 2026, Art. no. 122102.
[26] B. Hooi, H. A. Song, A. Beutel, N. Shah, K. Shin, and C. Faloutsos, "FRAUDAR: Bounding graph fraud in the face of camouflage," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 895–904.
[27] Y. Liu et al., "Pick and choose: A GNN-based imbalanced learning approach for fraud detection," in *Proc. Web Conf.*, 2021, pp. 3168–3177.
[28] Z. Chai et al., "Can abnormality be detected by graph neural networks," in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 1945–1951.
[29] Z. Gong et al., "Beyond homophily: Robust graph anomaly detection via neural sparsification," in *Proc. Int. Joint Conf. Artif. Intell.*, 2023, pp. 2104–2113.
[30] W. Zhuo et al., "Partitioning message passing for graph fraud detection," in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 1–19.
[31] N. Chen et al., "Consistency training with learnable data augmentation for graph anomaly detection with limited supervision," in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 1–28.
[32] M. Jin, Y. Liu, Y. Zheng, L. Chi, Y.-F. Li, and S. Pan, "ANEMONE: Graph anomaly detection with multi-scale contrastive learning," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 3122–3126.
[33] F. Liu et al., "DAGAD: Data augmentation for graph anomaly detection," in *Proc. Int. Conf. Data Mining*, 2022, pp. 259–268.
[34] I. Goodfellow et al., "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 2672–2680.
[35] H. Wang et al., "GraphGAN: Graph representation learning with generative adversarial nets," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 2508–2515.
[36] W. Yu et al., "Learning deep network representations with adversarially regularized autoencoders," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 2663–2671.
[37] A. Bojchevski, O. Shchur, D. Zügner, and S. Günnemann, "NetGAN: Generating graphs via random walks," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 609–618.
[38] N. De Cao and T. Kipf, "MolGAN: An implicit generative model for small molecular graphs," 2018, *arXiv:1805.11973*.
[39] Z. Chen, B. Liu, M. Wang, P. Dai, J. Lv, and L. Bo, "Generative adversarial attributed network anomaly detection," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2020, pp. 1989–1992.
[40] L. Deng, D. Lian, Z. Huang, and E. Chen, "Graph convolutional adversarial networks for spatiotemporal anomaly detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 6, pp. 2416–2428, Jun. 2022.
[41] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," 2013, *arXiv:1308.3432*.
[42] Y. Rong, W. Huang, T. Xu, and J. Huang, "Dropedge: Towards deep graph convolutional networks on node classification," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–18.
[43] M. Wiatrak, S. V. Albrecht, and A. Nystrom, "Stabilizing generative adversarial networks: A survey," 2019, *arXiv:1910.00927*.
[44] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C.-J. Hsieh, "Cluster-GCN: An efficient algorithm for training deep and large graph convolutional networks," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 257–266.
[45] J. J. McAuley and J. Leskovec, "From amateurs to connoisseurs: Modeling the evolution of user expertise through online reviews," in *Proc. Int. Conf. World Wide Web*, 2013, pp. 897–908.
[46] E. Müller, P. I. Sánchez, Y. Mülle, and K. Böhm, "Ranking outlier nodes in subspaces of attributed graphs," in *Proc. Int. Conf. Data Eng. Workshops*, 2013, pp. 216–222.
[47] J. Tang, J. Zhang, L. Yao, J. Li, L. Zhang, and Z. Su, "Arnetminer: Extraction and mining of academic social networks," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2008, pp. 990–998.
[48] L. Tang and H. Liu, "Relational learning via latent social dimensions," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2009, pp. 817–826.
[49] X. Wang, L. Tang, H. Gao, and H. Liu, "Discovering overlapping groups in social media," in *Proc. IEEE Int. Conf. Data Mining*, 2010, pp. 569–578.
[50] K. Ding, J. Li, R. Bhanushali, and H. Liu, "Deep anomaly detection on attributed networks," in *Proc. SIAM Int. Conf. Data Mining*, 2019, pp. 594–602.
[51] K. Liu et al., "Bond: Benchmarking unsupervised outlier node detection on static attributed graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 27021–27035.
[52] M. Shi, Y. Tang, X. Zhu, D. Wilson, and J. Liu, "Multi-class imbalanced graph convolutional network learning," in *Proc. Int. Joint Conf. Artif. Intell.*, 2020, pp. 2879–2885.
[53] H. Zenati, C. S. Foo, B. Lecouat, G. Manek, and V. R. Chandrasekhar, "Efficient GAN-based anomaly detection," 2018, *arXiv:1802.06222*.
[54] L. Qu, H. Zhu, R. Zheng, Y. Shi, and H. Yin, "Imgagn: Imbalanced network embedding via generative adversarial graph networks," in *ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2021, pp. 1390–1398.
[55] X. Dong, X. Zhang, L. Chen, M. Yuan, and S. Wang, "SpaceGNN: Multi-space graph neural network for node anomaly detection with extremely limited labels," in *Proc. Int. Conf. Learn. Representations*, 2025, pp. 1–24.
[56] C. Fu, Z. He, X. Zhang, C. Xie, X. Jin, and Y. Yang, "DR-GGAD: Dual residual centering for mitigating anomaly non-discriminativity in generalist graph anomaly detection," in *Proc. Int. Conf. Learn. Representations*, 2026, pp. 71971–71997.
[57] Y. Liu et al., "Towards anomaly-aware pre-training and fine-tuning for graph anomaly detection," in *Proc. Int. Conf. Learn. Representations*, 2026, pp. 136630–136663.
[58] Y. Liu et al., "Mitigating homophily disparity in graph anomaly detection: A scalable and adaptive approach," in *Proc. ACM Web Conf.*, 2026, pp. 1127–1138.
[59] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.
[60] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 11, 2008, Art. no. 2579.
[61] W. Hu et al., "Open graph benchmark: Datasets for machine learning on graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 22118–22133.
[62] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1024–1034.



Mengyu Li received the MS degree in computer science from Jilin University, Changchun, China, in 2024. She has authored or coauthored papers in multiple top-tier conferences and journals, including WWW, SIGIR, AAAI, IJCAI, and ACM TKDD. Her research interests include graph learning and natural language processing.



Yonghao Liu (Member, IEEE) received the PhD degree in software engineering from Jilin University, Changchun, China, in 2026. He is currently an associate professor with the College of Computer Science and Technology, Jilin University, China. He has authored or coauthored papers in multiple top-tier conferences and journals, including Nature Methods, NeurIPS, KDD, WWW, SIGIR, AAAI, IJCAI, and ACM TKDD. His research interests include graph learning, bioinformatics, and natural language processing.



Ximing Li (Member, IEEE) received the PhD degree from Jilin University, China. He currently a full professor with Jilin University. He has authored or coauthored more than 70 papers at the competitive venues, including ICLR, ICML, ACL, IJCAI, AAAI, WWW, SIGIR, and CIKM. His research interests include data mining, machine learning, and natural language processing.



Lan Huang received the PhD degree from Jilin University, China, in 2003. She is currently a professor and PhD supervisor specializing in business intelligence. Her research interests include on business intelligence and social network mining, resulting in about 70 academic papers and fourteen software copyrights. She won awards like Jilin Province's Outstanding Youth Project Fund and led the Young and Middle-aged Leader and Innovation Team. Invited as a senior visitor to Italy's Trento University, she has led more than 10 projects funded by various sources, winning awards such as the Jilin Province Scientific and Technological Progress Award. Her team's software developments have significantly benefited cooperative and application enterprises economically.



Fausto Giunchiglia received the PhD degree in computer engineering from the University of Genoa, Faculty of Engineering. He is currently a full professor with the Faculty of Science, University of Trento, Italy. His research interests include artificial intelligence, formal methods, knowledge management and agent-oriented software engineering.



Yanchun Liang received the PhD degree in applied mathematics from Jilin University, Changchun, China, in 1997. He was a visiting scholar with Manchester University, U.K. from 1990 to 1991, a visiting professor with the National University of Singapore from 2000 to 2001, a guest professor with the Institute of High Performance Computing of Singapore from 2002 to 2004, a guest professor with Trento University, Italy, from 2006 to 2010, and a guest professor with the University of Missouri, USA, from 2011 to 2017. He is currently a professor with the College of Computer Science and Technology, Jilin University. He has authored or coauthored more than 400 papers. His research interests include computational intelligence, machine learning methods, text mining, MEMS modeling and bioinformatics. His research has been featured in *Bioinformatics*, *IEEE Transactions on Systems, Man, and Cybernetics*, *IEEE Transactions on Knowledge and Data Engineering*, the *Journal of Micromechanics and Microengineering*, *Physical Review E*, *Neural Computing & Applications*, *Smart Materials and Structures*, *Artificial Intelligence in Medicine*, and *Applied Artificial Intelligence*. He was the recipient of several grants from NSFC and the EU.



Xiaoyue Feng (Member, IEEE) received PhD degrees in computer science from Jilin University, Changchun, China, in 2008. She is currently a full professor with the College of Computer Science and Technology, Jilin University, China. She has authored or coauthored more than 50 papers in prestigious journals and conferences, with her research featured in venues such as *Nature Methods*, *IEEE Transactions on Knowledge and Data Engineering*, ACM TKDD, KDD, NeurIPS, and WWW. Her research interests include bioinformatics, machine learning, text mining, and their applications to real-world problems. She was the recipient of several research grants, including those from the National Natural Science Foundation of China.



Renchu Guan (Senior Member, IEEE) received the PhD degree from Jilin University, Changchun, China, in 2010. He is currently a full professor with the College of Computer Science and Technology, Jilin University. He has authored or coauthored more than 70 papers. His research has been featured in *Nature Methods*, *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Geoscience and Remote Sensing*, *Nature Communications*, KDD, ICDE, NeurIPS, and WWW. His research interests include machine learning, bioinformatics, and knowledge engineering. He was the recipient of several grants from NSFC.